

RAG ASSIGNMENT

Final Project Report

Project	Retrieval-Augmented Generation Application for PDF Question Answering
Architecture	Local Ollama Models + FAISS Vector Search + Conversational Memory
Generation Model	Llama 3.2
Evaluation Model	Qwen 2.5 3B
Embedding Model	nomic-embed-text

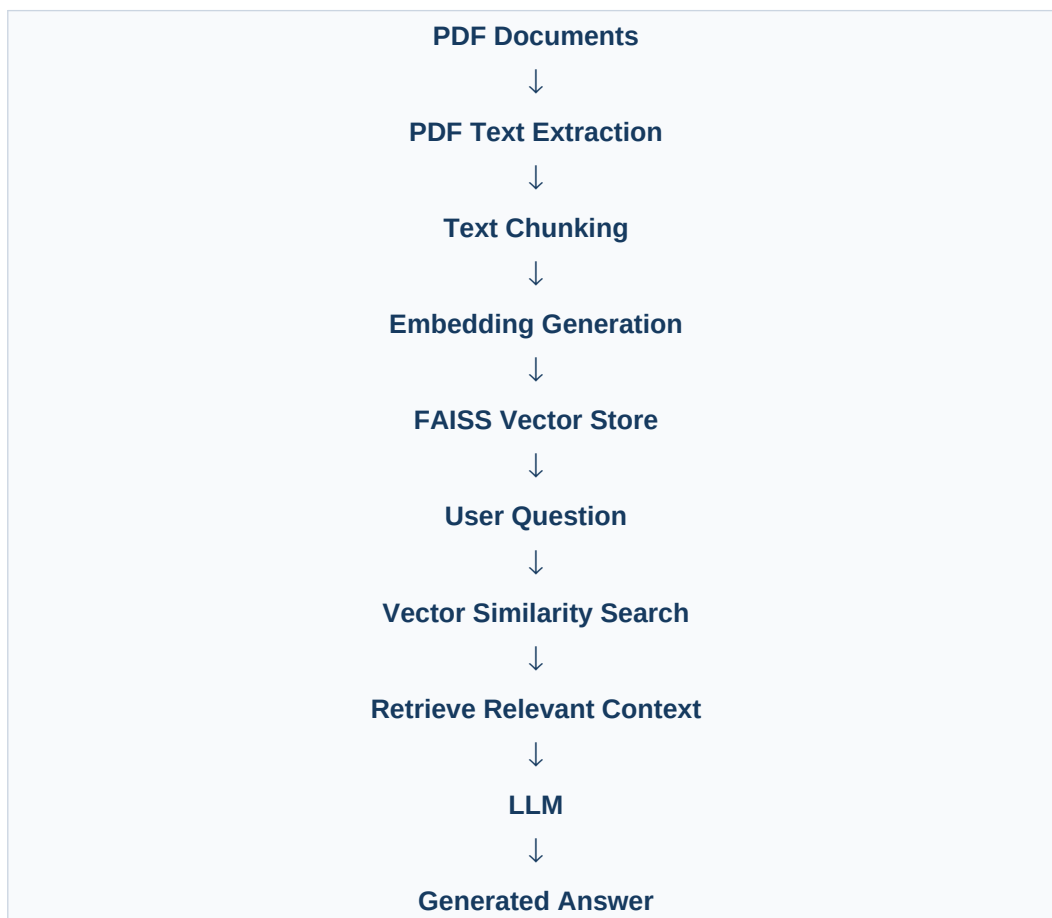
Submitted as part of the RAG Assignment

1. Overview

This project implements a complete Retrieval-Augmented Generation (RAG) application that answers user questions based on the content of PDF documents. Instead of relying only on the language model's general knowledge, the system retrieves relevant information from the supplied documents and provides that context to the language model before generating an answer.

The implementation uses a fully local architecture through Ollama. Llama 3.2 is used for answer generation, nomic-embed-text is used for semantic embeddings, FAISS is used for vector similarity search, and Qwen 2.5 3B is used as a local evaluator for RAGAS-based evaluation.

System Architecture



Approach

- Extract text from PDF documents.
- Split extracted text into manageable chunks.
- Generate vector embeddings for each chunk using nomic-embed-text.
- Store embeddings in a FAISS vector index.
- Convert user questions into embeddings and retrieve the most similar chunks.
- Provide retrieved context and the user question to Llama 3.2.
- Generate a grounded answer using the retrieved document context.

- Maintain recent conversation history using conversational memory.
- Evaluate the RAG pipeline using RAGAS metrics.

2. Technical Implementation

2.1 Data Extraction

PDF documents are placed inside the data/raw directory. The PDF loader reads each document, extracts text page by page, and prepares the extracted content for chunking. The extracted text is associated with metadata such as the source document name and page number so that retrieved information can later be traced back to its origin.

2.2 Text Chunking

The extracted text is split using RecursiveCharacterTextSplitter. Chunking is necessary because embedding and retrieval systems work more effectively with smaller units of text than with entire documents.

The project uses the following configuration:

```
chunk_size = 1000
chunk_overlap = 200
```

A chunk size of 1000 characters provides sufficient context while keeping each chunk reasonably focused. A chunk overlap of 200 characters helps preserve information across adjacent chunks and reduces the possibility of losing context at chunk boundaries.

2.3 Embedding Generation

Each text chunk is converted into a numerical vector representation using the nomic-embed-text model running locally through Ollama. The embedding process sends the text chunk to the local Ollama embedding API and stores the returned vector together with the chunk text and metadata.

- Embedding model: nomic-embed-text
- Execution environment: Local Ollama instance
- Input: Text chunks
- Output: Numerical embedding vectors

The embedded chunks are saved in data/processed/embedded_chunks.json.

2.4 Vector Database

FAISS is used as the vector similarity search engine. The project uses FAISS IndexFlatL2, which performs similarity search using L2 distance. All chunk embeddings are loaded into the FAISS index and saved for efficient retrieval.

- Vector search library: FAISS
- Index type: IndexFlatL2
- Stored index: data/processed/faiss_index.bin

- Purpose: Retrieve semantically similar document chunks for a user question

2.5 Retrieval Process

When a user asks a question, the question is converted into an embedding using the same embedding model. The resulting vector is searched against the FAISS index to identify the most relevant document chunks.

```
search(question, top_k=4)
```

The system retrieves the top four most relevant chunks. These chunks are combined into a structured document context containing source and page information.

2.6 Language Model Integration

Answer generation is performed using Llama 3.2 through the local Ollama API. The model receives a prompt containing the conversation history, retrieved document context, and current user question.

- Generation model: Llama 3.2
- Execution: Local Ollama API
- Input: Conversation history + retrieved context + user question
- Output: Generated answer

The prompt instructs the model to use the provided document context for document-related questions. If sufficient information is not available, the system responds that it does not have enough information in the provided documents.

2.7 Conversational Memory

The project includes a custom conversational memory component. The memory stores recent user questions and generated answers so that the system can understand follow-up questions and references to previous conversation turns.

```
memory = ConversationMemory(max_turns=4)
```

The most recent four conversation turns are included in the prompt. A `clear_conversation_memory` function is also available to reset the conversation state when required.

3. Evaluation

The RAG pipeline was evaluated using RAGAS. The evaluation framework examines different aspects of RAG quality, including whether answers are grounded in the retrieved context, whether answers match the reference answers, and whether the retrieved context contains relevant information.

3.1 Test Questions

#	Test Question
1	What is the Transformer architecture?
2	How does self-attention work?
3	What is multi-head attention?
4	Why are positional encodings used in Transformers?
5	What is the role of the encoder in a Transformer?
6	What is the role of the decoder in a Transformer?
7	What is causal self-attention?
8	What is a feed-forward network in a Transformer?
9	Why are residual connections used in Transformers?
10	What is layer normalization used for in Transformers?

3.2 Evaluation Framework

Evaluation data is generated by sending the predefined test questions through the RAG pipeline. For each question, the system stores the question, generated answer, retrieved contexts, and reference answer. The generated evaluation data is saved in `data/evaluation_results.json`.

3.3 Evaluation Models

- Evaluator LLM: Qwen 2.5 3B running locally through Ollama.
- Embedding model: nomic-embed-text.
- Evaluation framework: RAGAS.

3.4 RAGAS Metrics

Metric	Purpose
Faithfulness	Measures whether the generated answer is supported by the retrieved context. A high score indicates that the answer is grounded in the retrieved context.
Answer Correctness	Measures how closely the generated answer matches the reference or ground-truth answer.
Context Recall	Measures whether the retrieved context contains sufficient information needed to answer the question.
Context Precision	Measures how relevant the retrieved chunks are to the question and expected answer.

The evaluation framework was initially executed against all 10 questions. During cloud-based evaluation, API token limits and rate-limit errors occurred. The final local evaluation run successfully evaluated two questions using the local Qwen 3B evaluator. The results below therefore represent the completed two-question evaluation run.

4. Results & Discussion

4.1 Evaluation Results

Metric	Score	Successful Evaluations
Faithfulness	100.00%	2/2
Answer Correctness	53.64%	2/2
Context Recall	66.67%	2/2
Context Precision	100.00%	2/2

4.2 Analysis of Results

Faithfulness – 100.00%

The generated answers were fully supported by the retrieved document context for the successfully evaluated questions. This is a strong result because it indicates that the RAG pipeline is effectively grounding answers in retrieved information and reducing hallucination.

Context Precision – 100.00%

The retrieved chunks were highly relevant to the questions. Retrieving the top four chunks using semantic similarity provided useful context without introducing significant irrelevant information.

Answer Correctness – 53.64%

Answer correctness was moderate. Although the answers were grounded in the retrieved context, they did not always closely match the expected reference answers. This can occur because an answer may be factually reasonable but phrased differently, may omit important details, or may not cover all information present in the reference answer.

Context Recall – 66.67%

The retrieved context contained a significant portion of the information required to answer the questions, but some relevant information was missing. This suggests that retrieval could be improved further through chunking adjustments, retrieval tuning, or reranking.

4.3 Challenges Faced

- Cloud-based evaluation encountered API rate limits after a large number of evaluation requests.
- Some evaluator requests failed because the configured maximum token limit was insufficient.
- RAGAS evaluation can be computationally expensive because multiple LLM-based metric evaluations are performed for each question.
- Local evaluation avoids cloud API limits but can be slower depending on available CPU, RAM, and model size.
- Evaluation results are sensitive to the quality and wording of the reference answers.
- Only two questions completed successfully in the final local evaluation run, so the numerical results should be interpreted as an initial evaluation rather than a statistically comprehensive benchmark.

4.4 Potential Improvements

- Experiment with smaller or larger chunk sizes to improve retrieval quality.
- Tune chunk overlap to preserve relationships between adjacent sections.
- Experiment with different top_k values during retrieval.

- Add a reranking model after FAISS retrieval.
- Implement hybrid search combining keyword search and vector similarity search.
- Improve prompt instructions to encourage more complete and concise answers.
- Experiment with stronger local generation models.
- Evaluate all 10 test questions after managing rate limits and token configuration.
- Create more detailed reference answers for improved answer correctness evaluation.
- Test alternative embedding models.

5. Conclusion

This project successfully demonstrates the implementation of an end-to-end Retrieval-Augmented Generation application for answering questions from PDF documents. The system extracts text from PDFs, splits the content into chunks, generates embeddings locally using nomic-embed-text, stores vectors in FAISS, retrieves relevant context through similarity search, and uses Llama 3.2 to generate answers.

The application also implements conversational memory by retaining the most recent four conversation turns, allowing it to support follow-up questions. RAGAS was used to evaluate the quality of the pipeline using Faithfulness, Answer Correctness, Context Recall, and Context Precision metrics.

The completed local evaluation produced strong Faithfulness and Context Precision scores of 100%, demonstrating that the system was able to generate grounded answers from relevant retrieved information. Answer Correctness and Context Recall showed opportunities for improvement, particularly in retrieval coverage and answer completeness.

Overall, the project demonstrates the complete lifecycle of building, running, and evaluating a local RAG application. Future improvements such as hybrid retrieval, reranking, improved chunking strategies, stronger language models, and evaluation across the complete 10-question dataset can further improve overall performance.